



Tensor Contraction Interface and Block Swizzling

Max Koch

Optimization Space

- There exist different implementations for the contraction `abklxy, cklyz -> abcxz`

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     for k_it in range(k):
7         for l_it in range(l):
8             # Matmul shape = (x, z, y)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (bc)
5
6     for a_it in range(a):
7         for k_it in range(k):
8             for l_it in range(l):
9                 # Matmul shape = (x, z, y)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     for k_it in range(k):
7         # Matmul shape = (x, z, y * l)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     # Matmul shape = (x, z, y * l * k)
```

Optimization Space

- Which dimensions should be parallelized?

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     for k_it in range(k):
7         for l_it in range(l):
8             # Matmul shape = (x, z, y)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (bc)
5
6     for a_it in range(a):
7         for k_it in range(k):
8             for l_it in range(l):
9                 # Matmul shape = (x, z, y)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     for k_it in range(k):
7         # Matmul shape = (x, z, y * l)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     # Matmul shape = (x, z, y * l * k)
```

Optimization Space

- Which dimensions should be sequentialized?

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     for k_it in range(k):
7         for l_it in range(l):
8             # Matmul shape = (x, z, y)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (bc)
5
6     for a_it in range(a):
7         for k_it in range(k):
8             for l_it in range(l):
9                 # Matmul shape = (x, z, y)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     for k_it in range(k):
7         # Matmul shape = (x, z, y * l)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     # Matmul shape = (x, z, y * l * k)
```

Optimization Space

- Which dimensions should be used for the (B)GEMM primitive?

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     for k_it in range(k):
7         for l_it in range(l):
8             # Matmul shape = (x, z, y)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (bc)
5
6     for a_it in range(a):
7         for k_it in range(k):
8             for l_it in range(l):
9                 # Matmul shape = (x, z, y)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     for k_it in range(k):
7         # Matmul shape = (x, z, y * l)
```

```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4     # Decompose pid -> (abc)
5
6     # Matmul shape = (x, z, y * l * k)
```

Contraction Pattern

- General pattern for contraction kernels:

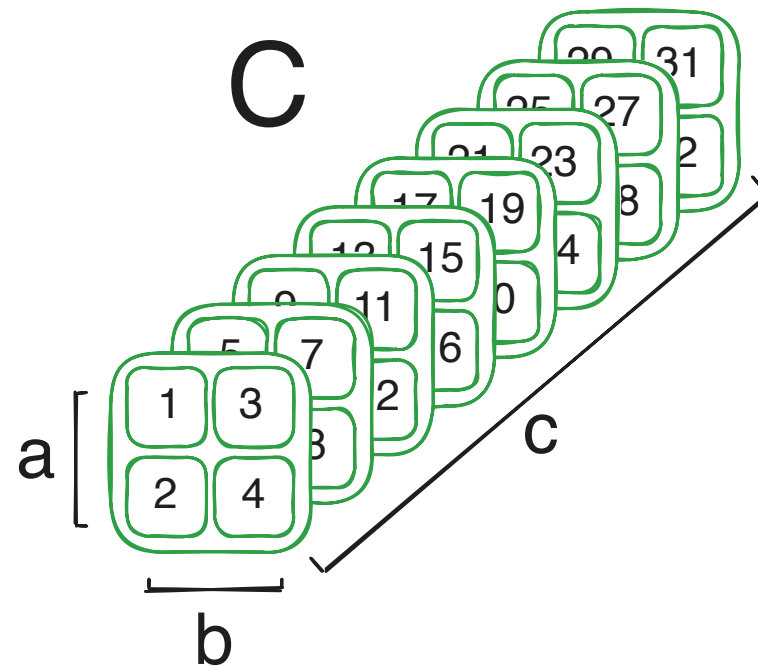
```
1 @ct.kernel
2 def contraction(A, B, C):
3     pid = ct.bid(0)
4
5     # Decompose pid -> retrieve dimension indices for parallel dimensions
6     par_0 = ...
7     par_1 = ...
8     [...]
9     par_n = ...
10
11     # Loop over all sequential dimensions (including contraction dimensions)
12     for it_seq_0 in size_dim_0:
13         for it_seq_1 in size_dim_1:
14             [...]
15             for it_seq_n in size_dim_n:
16                 # Compute tile of C using a (B)GEMM
17
18     # Store tile of C to global memory
```

Optimization Space

- Different implementations have different performance characteristics
- Optimization space is large and complex
 - Which dimensions should be parallelized/sequentialized?
 - Which dimensions should be used for the (B)GEMM primitive?
 -
 -

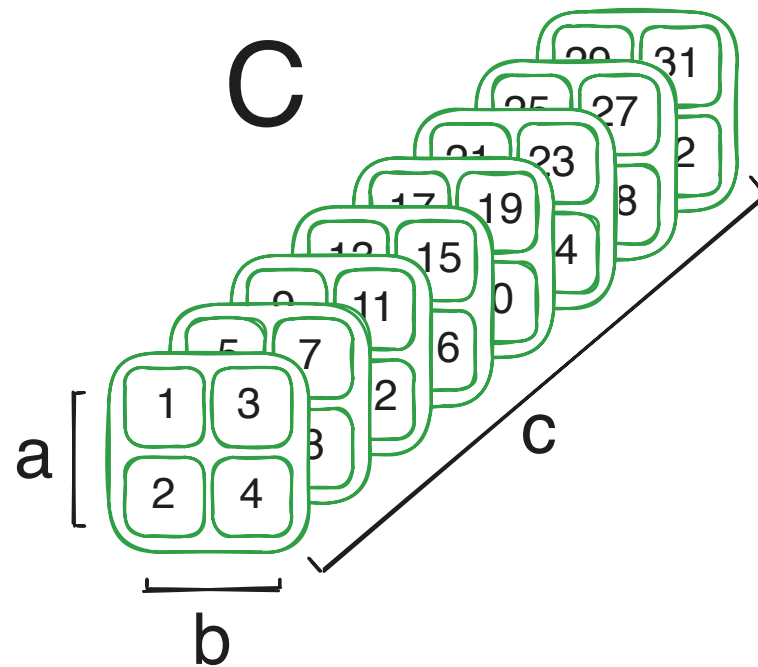
Optimization Space

- Different implementations have different performance characteristics
- Optimization space is large and complex
 - Which dimensions should be parallelized/sequentialized?
 - Which dimensions should be used for the (B)GEMM primitive?
 - How should the BIDs be mapped to the parallel dimensions?
-



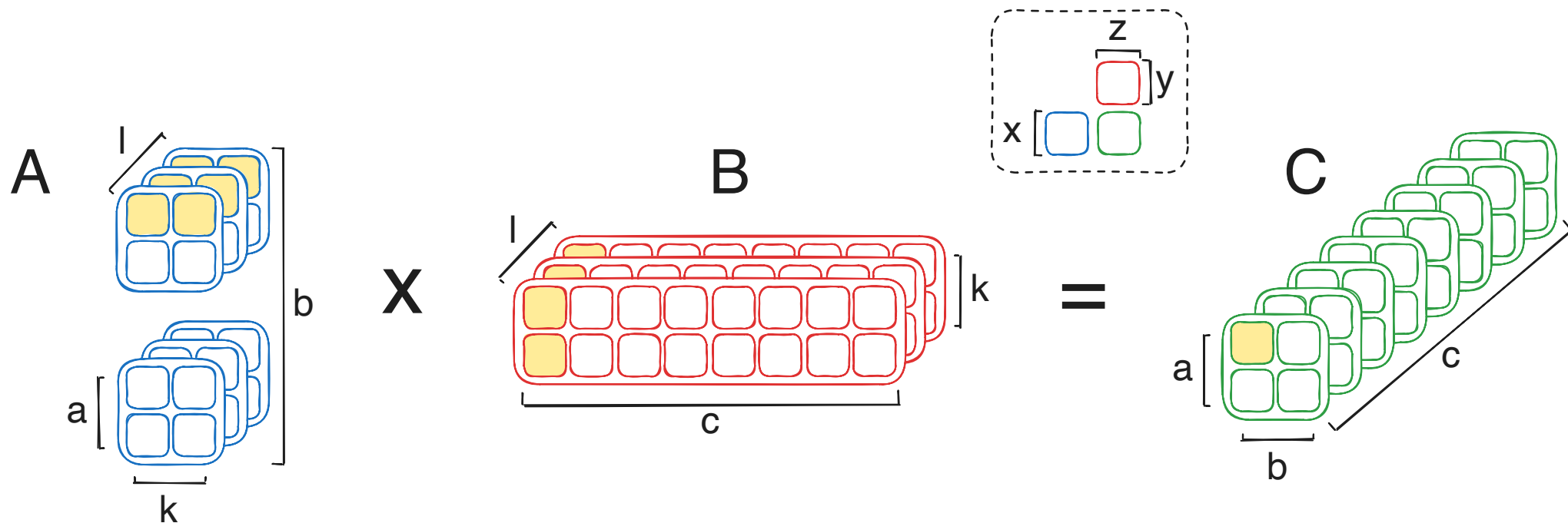
Optimization Space

- Different implementations have different performance characteristics
- Optimization space is large and complex
 - Which dimensions should be parallelized/sequentialized?
 - Which dimensions should be used for the (B)GEMM primitive?
 - How should the BIDs be mapped to the parallel dimensions?
 - What dimensions should be split/fused?



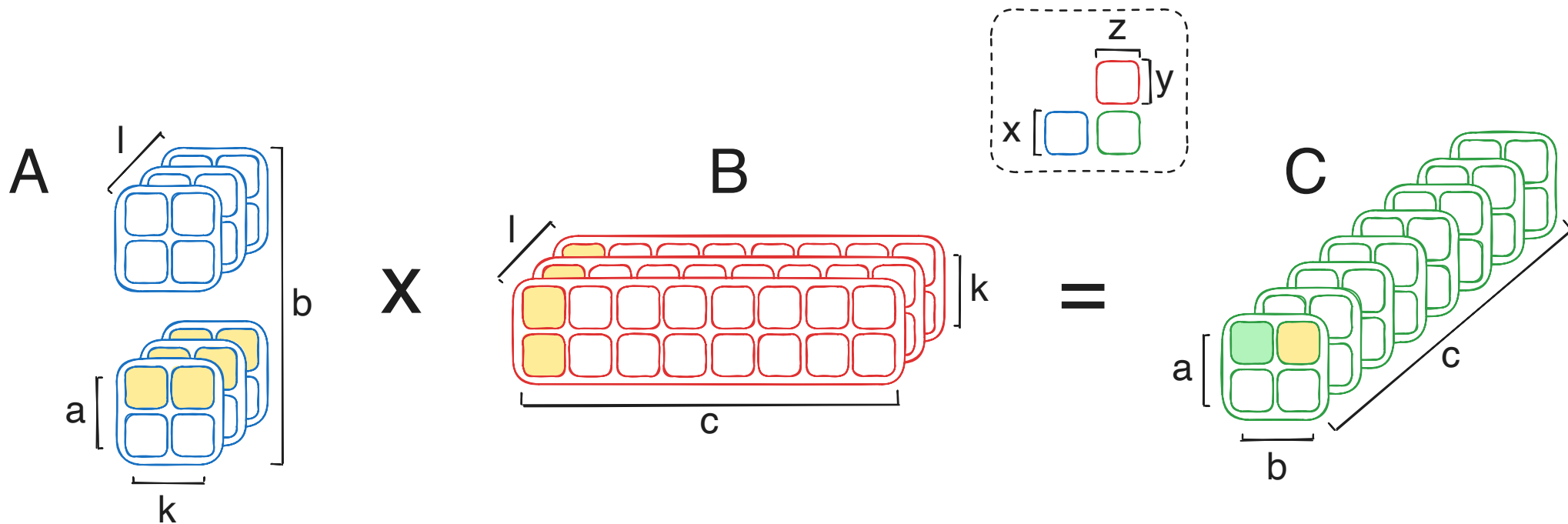
Block ID Mapping

- We know which tiles we need to compute one output tile



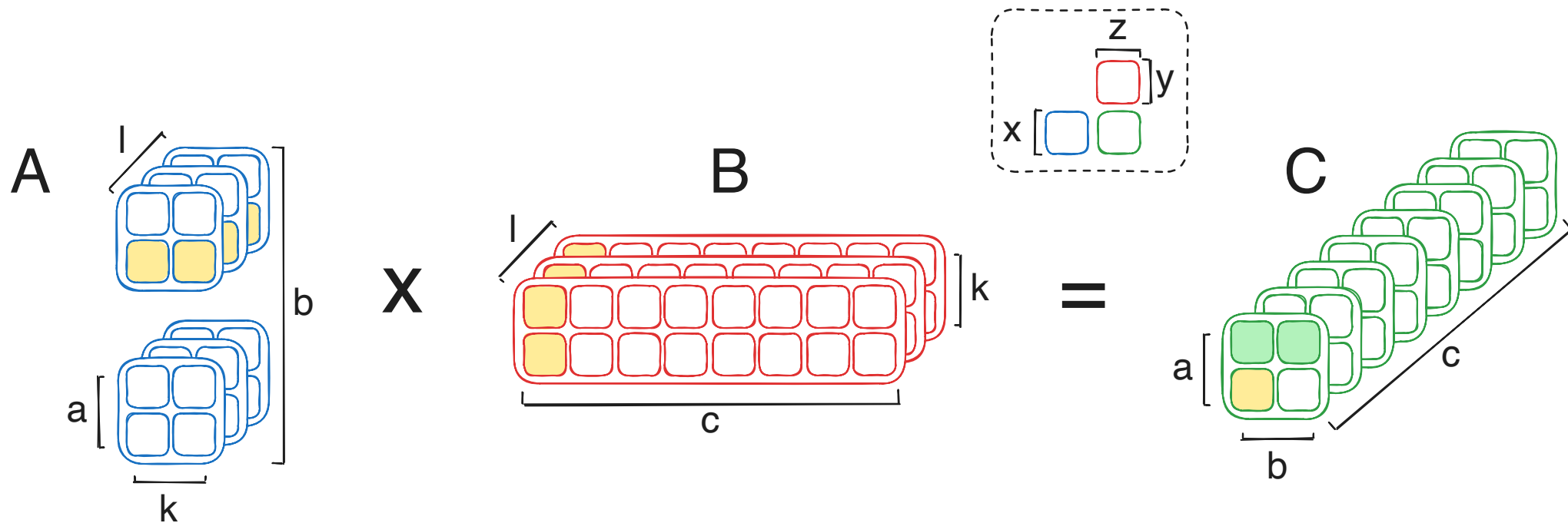
Block ID Mapping

- We know which tiles we need to compute one output tile



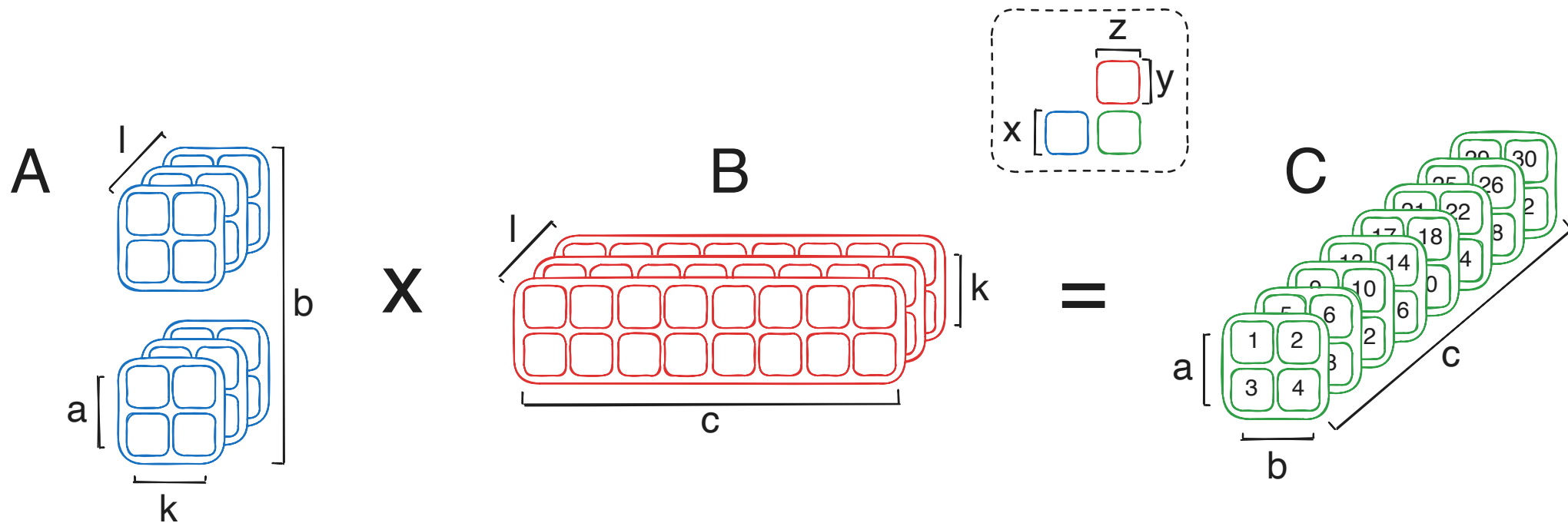
Block ID Mapping

- We know which tiles we need to compute one output tile



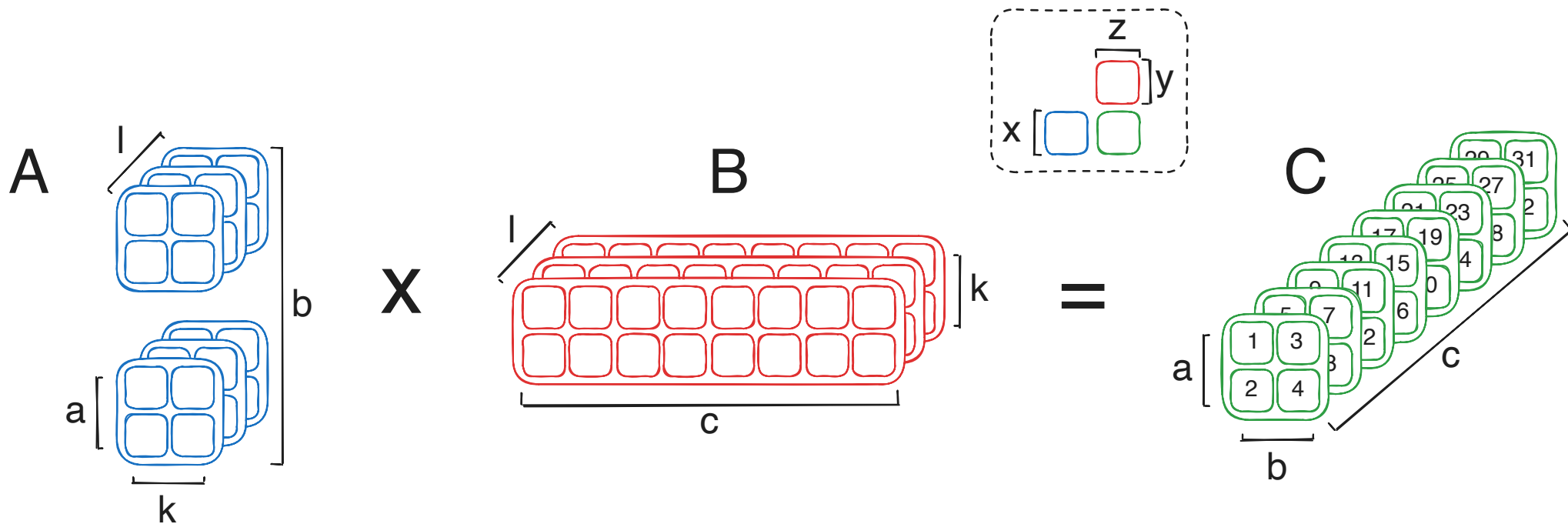
Block ID Mapping

- Row major order of parallel dimensions for BID mapping: **cab**



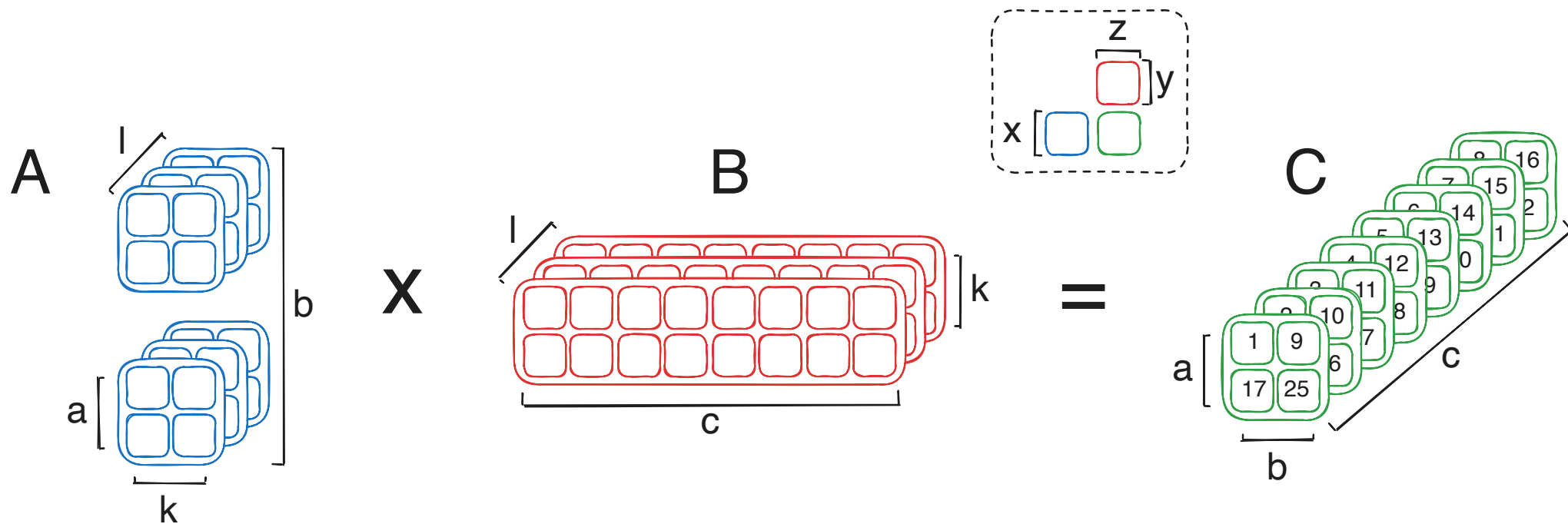
Block ID Mapping

- Row major order of parallel dimensions for BID mapping: **cba**



Block ID Mapping

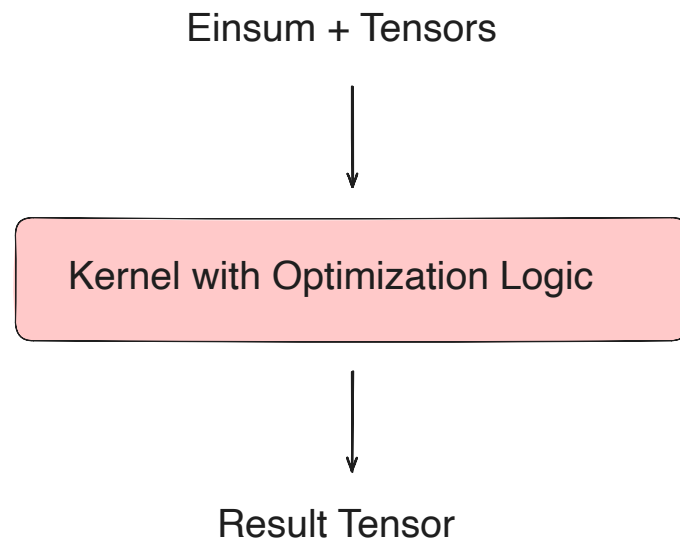
- Row major order of parallel dimensions for BID mapping: **abc**



Optimization Logic inside the Kernel

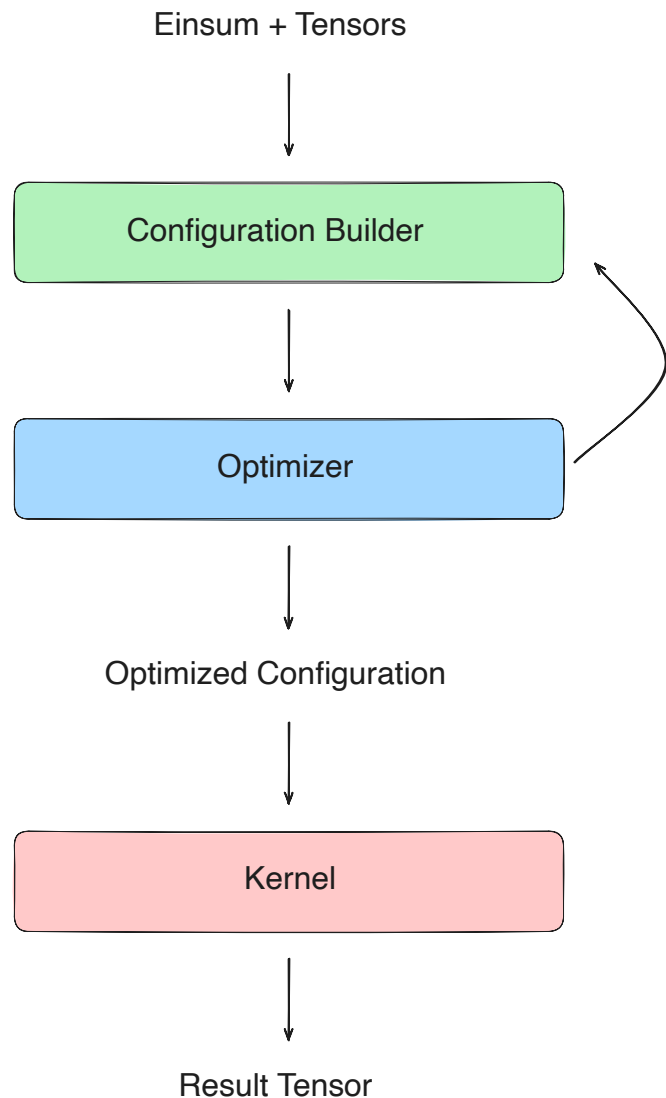
- L2 reuse logic (BID swizzling) for tensor contractions can get quite complicated
- Heavy logic inside the kernel is needed to determine which tiles to operate on
- For each tensor contraction shape, the underlying logic can be different

→ **Programming and runtime overhead**



Optimization Logic outside the Kernel

- Provide a high-level contraction interface that allows tensor contraction configurations to be easily expressed
- Given an abstract contraction, it should be easy to write (or generate) a kernel that implements the desired contraction
- Optimizing the configuration should be easily possible without having to think about complicated index math



Building the Interface

wxy, wyz -> wxz

- Let's first define some fields that don't change during optimization

```
1 # one possible configuration:
2
3 config.data_type = config.float16
4 config.prim_main = config.GEMM
5 config.prim_last = config.none
6 config.prim_first = config.zero
7
8
9
10
11
12
13
14
```

Building the Interface

wxy, wyz -> wxz

- Let's first define some fields that don't change during optimization

```
1 # one possible configuration:
2
3 config.data_type = config.float16
4 config.prim_main = config.GEMM
5 config.prim_last = config.none
6 config.prim_first = config.zero
7
8
9 # or another possible configuration:
10
11 config.data_type = config.float32
12 config.prim_main = config.BGEMM
13 config.prim_last = config.elwise_mul
14 config.prim_first = config.zero
```

Building the Interface

wxy, wyz -> wxz

- Let's first define some fields that don't change during optimization

```
1 # one possible configuration:
2
3 config.data_type = config.float16
4 config.prim_main = config.GEMM
5 config.prim_last = config.none
6 config.prim_first = config.zero
7
8
9 # or another possible configuration:
10
11 config.data_type = config.float32
12 config.prim_main = config.BGEMM
13 config.prim_last = config.elwise_mul
14 config.prim_first = config.zero
```

Building the Interface

wvxy, wvyz -> wxz

- Now we can define the fields regarding the dimensions

1 #	w,	v,	x,	y,	z
2					
3					
4					
5					
6					
7					
8					
9					
10					

Building the Interface

wvxy, wvyz -> wxz

- Now we can define the fields regarding the dimensions

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c,   config.k,   config.m,   config.k,   config.n ]
4
5
6
7
8
9
10
```

Building the Interface

wvxy, wvyz -> wxz

- Now we can define the fields regarding the dimensions

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c,  config.k,  config.m,  config.k,  config.n ]
4 config.exec_types = [config.seq, config.seq, config.seq, config.seq, config.seq]
5
6
7
8
9
10
```

Building the Interface

wvxy, wvyz -> wxz

- Now we can define the fields regarding the dimensions

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c,  config.k,  config.m,  config.k,  config.n ]
4 config.exec_types = [config.seq, config.seq, config.seq, config.seq, config.seq]
5
6 config.dim_sizes = [4,           2,           4096,           4096,           4096           ]
7
8
9
10
```

Building the Interface

wvxy, wvyz -> wxz

- Now we can define the fields regarding the dimensions

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c,  config.k,  config.m,  config.k,  config.n ]
4 config.exec_types = [config.seq, config.seq, config.seq, config.seq, config.seq]
5
6 config.dim_sizes = [4,           2,           4096,           4096,           4096           ]
7
8 config.strides   = [[33554432,  16777216,  4096,           1,           0           ],
9
10
```

Building the Interface

wvxy, wvyz -> wxz

- Now we can define the fields regarding the dimensions

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c,  config.k,  config.m,  config.k,  config.n ]
4 config.exec_types = [config.seq, config.seq, config.seq, config.seq, config.seq]
5
6 config.dim_sizes = [4,           2,           4096,      4096,      4096      ]
7
8 config.strides   = [[33554432,  16777216,  4096,        1,          0          ],
9                    [33554432,  16777216,  0,          4096,      1          ],
10
```

Building the Interface

wvxy, wvyz -> wxz

- Now we can define the fields regarding the dimensions

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c, config.k, config.m, config.k, config.n ]
4 config.exec_types = [config.seq, config.seq, config.seq, config.seq, config.seq]
5
6 config.dim_sizes = [4,           2,           4096,           4096,           4096           ]
7
8 config.strides   = [[33554432, 16777216, 4096,           1,           0           ],
9                    [33554432, 16777216, 0,           4096,           1           ],
10                   [16777216, 0,           4096,           0,           1           ]]
```

Building the Interface

wvxy, wvyz -> wxz

- Now we can define the fields regarding the dimensions

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c,  config.k,  config.m,  config.k,  config.n ]
4 config.exec_types = [config.seq, config.seq, config.seq, config.seq, config.seq]
5
6 config.dim_sizes = [4,           2,           4096,           4096,           4096           ]
7
8 config.strides    = [[33554432,  16777216,  4096,           1,           0           ],
9                      [33554432,  16777216,  0,           4096,           1           ],
10                     [16777216,  0,           4096,           0,           1           ]]
```

Optimize Configuration: Exec Types

wvxy, wvyz -> wxz

- We can set `config.exec_types` to choose parallel/sequential dimensions

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c,  config.k,  config.m,  config.k,  config.n  ]
4 config.exec_types = [config.seq, config.seq, config.prim, config.prim, config.prim]
5
6 config.dim_sizes = [4,           2,           4096,           4096,           4096           ]
7
8 config.strides    = [[33554432,  16777216,  4096,           1,           0           ],
9                      [33554432,  16777216,  0,           4096,           1           ],
10                     [16777216,  0,           4096,           0,           1           ]]
```

Optimize Configuration: Exec Types

wvxy, wvyz -> wxz

- We can set `config.exec_types` to choose parallel/sequential dimensions

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c,  config.k,  config.m,  config.k,  config.n  ]
4 config.exec_types = [config.par, config.seq, config.prim, config.prim, config.prim]
5
6 config.dim_sizes = [4,           2,           4096,           4096,           4096           ]
7
8 config.strides    = [[33554432,  16777216,  4096,           1,           0           ],
9                      [33554432,  16777216,  0,           4096,           1           ],
10                     [16777216,  0,           4096,           0,           1           ]]
```

Optimize Configuration: Split Dimension

wvxy, wvyz -> wxz

New tensor view: abvxy, abvyz -> abxz

- It is possible to split dimensions inside the configuration, e.g., split w into a and b

```
1 # w,          v,          x,          y,          z
2
3 [config.c,    config.k,    config.m,    config.k,    config.n    ]
4 [config.par,  config.seq,  config.prim, config.prim, config.prim]
5
6 # dim_sizes
7 [4,          2,          4096,          4096,          4096          ]
8
9 # strides
10 [[33554432,   16777216,   4096,          1,          0          ],
11  [33554432,   16777216,   0,          4096,          1          ],
12  [16777216,   0,          4096,          0,          1          ]]
```

Optimize Configuration: Split Dimension

wvxy, wvyz -> wxz

New tensor view: abvxy, abvyz -> abxz

- It is possible to split dimensions inside the configuration, e.g., split w into a and b

```
1 # a,          b,          v,          x,          y,          z
2
3 [config.c,    config.c,    config.k,    config.m,    config.k,    config.n    ]
4 [config.par,  config.seq,  config.seq, config.prim, config.prim, config.prim]
5
6 # dim_sizes
7 [2,          2,          2,          4096,        4096,        4096        ]
8
9 # strides
10 [[67108864,   33554432,   16777216,   4096,        1,          0          ],
11  [67108864,   33554432,   16777216,   0,          4096,        1          ],
12  [33554432,   16777216,   0,          4096,        0,          1          ]]
```

Optimize Configuration: Fuse Dimensions

abvxy, abvyz -> abxz

New tensor view: wvxy, wvyz -> wxz

- It is possible to fuse dimensions inside the configuration, e.g., fuse **a** and **b** into **w**
- To be able to fuse dimensions, the dimensions have to be adjacent in every tensor they appear in, e.g., **a** and **b** are adjacent in all three tensors
- Adjacency can be checked via dimension sizes and strides

```
1 #           w,           v,           x,           y,           z
2
3 config.dim_types = [config.c, config.k, config.m, config.k, config.n ]
4 config.exec_types = [config.seq, config.seq, config.prim, config.prim, config.prim]
5
6 config.dim_sizes = [4,           2,           4096,           4096,           4096           ]
7
8 config.strides   = [[33554432, 16777216, 4096,           1,           0           ],
9                    [33554432, 16777216, 0,           4096,           1           ],
10                   [16777216, 0,           4096,           0,           1           ]]
```

Optimize Configuration: Permute Dimensions

abvxy, abvyz -> abxz

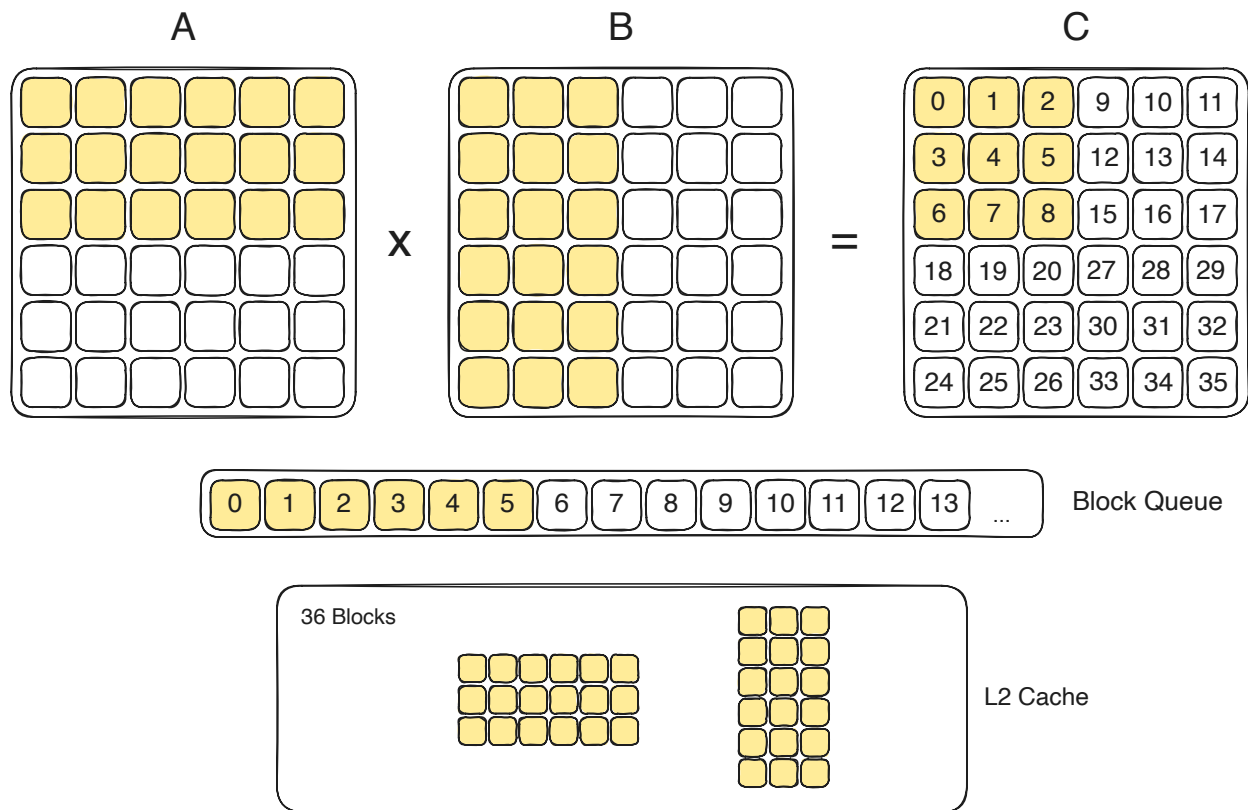
View does not change

- Execution order is defined by the order of dimensions in the configuration, i.e., the leftmost dimension corresponds to the slowest-changing index
- Permuting dimensions in the configuration changes the execution order, i.e., the BID mapping and loop order

```
1 # b,          a,          v,          x,          y,          z
2 [config.c,    config.c,    config.k,    config.m,    config.k,    config.n  ]
3 [config.par,  config.seq,  config.seq, config.prim, config.prim, config.prim]
4
5 # dim_sizes
6 [2,          2,          2,          4096,        4096,        4096      ]
7
8 # strides
9 [[33554432,   67108864,   16777216,   4096,        1,          0          ],
10 [33554432,   67108864,   16777216,   0,          4096,        1          ],
11 [16777216,   33554432,   0,          4096,        0,          1          ]]
```

Optimization Example: L2 Reuse

- We can achieve the swizzled matrix multiplication from lecture 3 by optimizing the configuration of a non-swizzled matrix multiplication $mk, kn \rightarrow mn$



Optimization Example: L2 Reuse

- Since the configuration's execution order is defined from right to left, we can achieve L2 reuse through optimizing the configuration (equivalent to swizzling inside the kernel):
 1. Fuse and split dimensions to retrieve dimensions of a specific target size for L2 reuse
 2. Move those dimensions to the right inside the configuration (through permutation)
 3. Use the other dimensions (on the left) for parallelization or looping inside the kernel
- General pattern for L2 reuse optimization:

```
1 # general L2 reuse pattern
2 # =====
3
4 # m_l2, n_l2 used for L2 reuse,
5 # prim_m, prim_n, prim_k used for the GEMM primitive
6 # [...] - all other dimensions
7
8 config.dim_sizes = [ [...], |m_l2|, |n_l2|, |prim_m|, |prim_n|, |prim_k| ]
```